

UVa10808 Rational Resistors

题目链接

UVa10808

题意

给你一个包含n个结点，m条导线的电阻网络，求结点a和b之间的等效电阻。a和b的等效电阻等于有一安培电流从a流入b流出时，ab之间的电压值。注意两个结点之间可以有多条导线相连，一条导线的两端也可以是同一个结点。输出用最简分数表示（无穷大用1/0 表示）。

分析

根据基尔霍夫定律列方程组求解即可。要注意两点：1、分数运算下为了避免long long溢出，高斯消元需要倒着枚举（尽量优先使用未参与消元的方程作为基准，或者使用分数的分子分母最大值最小的方程）2、源点汇点方程的处理也需要注意选更好的方式

```
for (int i=0; i<n; ++i) a[t][i] = a[i][n] = fract();
a[t][t] = a[s][n] = fract(1, 1);
// 看起来和上面两行等价，实际上上面的写法更好
// for (int i=0; i<n; ++i) {
//     a[i][t] = fract();
//     a[i][n] = i==s ? fract(1, 1) : (i==t ? fract(-1, 1) : fract());
// }
for (int i=0; i<n; ++i) {
    for (int j=n-1; j>=i; --j) if (a[j][i].n) { // 倒序是为了避免long long溢出
        if (j > i) for (int k=i; k<=n; ++k) f = a[i][k], a[i][k] = a[j][k], a[j][k] = f;
        for (j=0; j<n; ++j) if (j!=i && a[j][i].n) {
            f = a[j][i].div(a[i][i]); a[j][i] = fract();
            for (int k=n; k>i; --k) a[j][k].sub(a[i][k].mul(f));
        }
        break;
    }
}
```

AC代码

```
#include <iostream>
#include <cstring>
#include <numeric>
using namespace std;

#define N 33
struct fract {
    long long n, d;
    fract(long long n = 0, long long d = 1): n(n), d(d) {simplify();}
    void simplify() {
        long long g = gcd(n, d);
        n /= g; d /= g;
        if (d < 0) n=-n, d=-d;
    }
    void add(const fract& f) {
        if (f.n) {
            n = n*f.d + d*f.n; d *= f.d;
            simplify();
        }
    }
    void sub(const fract& f) {
        if (f.n) {
            n = n*f.d - d*f.n; d *= f.d;
            simplify();
        }
    }
    fract mul(const fract& f) {
        return fract(n*f.n, d*f.d);
    }
    fract div(const fract& f) {
        return fract(n*f.d, d*f.n);
    }
} a[N][N], b[N][N], f;
int p[N], n, m, t, q, kase = 0;

int find(int x) {
    return x == p[x] ? x : p[x] = find(p[x]);
}

void solve() {
    cin >> n >> m;
    for (int i=0; i<n; ++i) {
        p[i] = i;
        for (int j=0; j<n; ++j) b[i][j] = fract();
    }
    while (m--) {
        int u, v, r; cin >> u >> v >> r;
        if (u != v) {
            p[find(u)] = find(v);
            f.n = 1; f.d = r;
            b[u][u].add(f); b[u][v].sub(f);
            b[v][v].add(f); b[v][u].sub(f);
        }
    }
    cout << "Case #" << ++kase << ':' << endl;
    cin >> q;
    while (q--) {
        memcpy(a, b, sizeof(b));
        int s, t; cin >> s >> t;
        if (s == t) {
            cout << "Resistance between " << s << " and " << t << " is 0/1" << endl;
            continue;
        }
        if (find(s) != find(t)) {
            cout << "Resistance between " << s << " and " << t << " is 1/0" << endl;
            continue;
        }
        for (int i=0; i<n; ++i) a[t][i] = a[i][n] = fract();
        a[t][t] = a[s][n] = fract(1, 1);
        // 看起来和上面两行等价，实际上上面的写法更好
        // for (int i=0; i<n; ++i) {
        //     a[i][t] = fract();
        //     a[i][n] = i==s ? fract(1, 1) : (i==t ? fract(-1, 1) : fract());
        // }
        for (int i=0; i<n; ++i) {
            for (int j=n-1; j>=i; --j) if (a[j][i].n) { // 倒序是为了避免long long溢出
                if (j > i) for (int k=i; k<=n; ++k) f = a[i][k], a[i][k] = a[j][k], a[j][k] = f;
                for (j=0; j<n; ++j) if (j!=i && a[j][i].n) {
                    f = a[j][i].div(a[i][i]); a[j][i] = fract();
                    for (int k=n; k>i; --k) a[j][k].sub(a[i][k].mul(f));
                }
                break;
            }
        }
        f = a[s][n].div(a[s][s]);
        cout << "Resistance between " << s << " and " << t << " is " << f.n << "/" << f.d << endl;
    }
    cout << endl;
}

int main() {
    cin >> t;
    while (t--) solve();
}
```